

Exploring The Weight Space of a CNN-Classifier

1 Riemannian Manifolds

As stated in [3], Deep Learning Models' weights space can be considered a Riemannian Manifold associated to a Riemannian metric that allows to compute for each and every pair of points in that space the distance and the angle as they lied in a flat space as long as the "zooming" is enough. To better understand it let's break down what Riemannian Manifolds are:

Riemannian Manifolds are Curve spaces that can be viewed as flat 'zooming enough', imagine the planet Earth, we know it is a geosphere with many imperfections, but if we look at a human scale it seems flat and we can even use any 2D flat space metric to measure distances and Pythagorean distances. Now, a Riemannian Manifold is also associated with a metric that gives the angle and distance for each pair of points in that Manifold. Imagine walking with a mini-ruler and a mini-compass giving you the metric of distance and angle for the entire neighborhood of your position and you are lost in this vast space seemingly flatten but curve in reality, that's a riemannian manifold.

To give a more mathematical intuition of what Riemannian manifolds are, think about integrals: calculating a Riemannian integrals means to consider the $\Delta x \rightarrow 0$. In that tendency we zoom in the curve we are computing and we notice it starts becoming flat to a point where we can consider the area under that infinitesimal space as a rectangle. Here we can use the $h \cdot w$ to compute the area of the rectangle, for then moving to the next dx so we sum the previous computation to the new one and we are basically walking to the curve in a direction summing each infinitesimal rectangle to generate the overall curve. Riemannian Manifolds extend this concept to an entire N-Dimensional space: we have a very huge curve surface, when we zoom in in the infinitesimal surroundings of a point, we can consider it as flat, so we can consider the tangent flat space to the curve (flat means also linear so we can apply all the linear distance metrics for that ND-space). Now if we move of small steps dx around, we can compute the turning angle and the distance we traveled and repeat the flattening approximation on the new point, summing all of that we can get the overall curvature of the space and the distance traveled, while also spotting the geodesics and hence the shortest paths.

1.1 Geodesics

In a Riemannian Manifold a core concept are the Geodesics which are the shortest paths between two different points in that space. To understand what a Geodesic is, imagine a flat paper, draw two points on it, then connect them with a line. The line is straight, and thus is the shortest path to connect the two points (we are not curving anywhere so we are not artificially increasing the distance). Now bend the paper as you like to create the Riemannian Manifold, and the line will be the Geodesic.

1.2 Weight Space as Riemannian Manifold

Have you noticed that when training any Deep Neural Network, there are a lot of very low value weights and a just some high value weights? When a DNN gets trained there are weights that impact the loss more than others, this suggests that the weight-space can be traversed in many directions, each one impacting the loss function in a different way. The delta of the loss function while moving in the weight space can be considered as the Riemannian Metric associated to the Manifold. The entire collection of possible weight configurations forms a high-dimensional space. Each point in this space corresponds to a specific neural network. For such space euclidean distance could lead to catastrophic results as changing specific weights can have much more impact than other weights. This tells that the space is highly non-euclidean and the direction we take (the set of weights we decide to change) do not behave linearly. Another core aspect is that we might scale all weights of the network and obtain identical neural network (think of Batch-Normalization, we do not care if the neuron weights are in the scale of $1e2$ or $1e-4$ as long as they have that specific configuration), which would basically have the effect of resizing the weight-space without really changing it. Riemannian Manifold use a metric of distancing and angles to actually have a uniform metric that allows us to reliably calculate the distance and angle. This metric is the core aspect to transform the weight-space into a Riemannian Manifold. Imagine having a ruler that changes its sensitivity depending on the space features and direction, if we could have that we can consider the neighborhood as flat and use that metric as it was the euclidean distance, without it we are blind.

1.3 Riemannian Metric

As stated in the previous section, without a Riemann Metric we cannot transform the weight-space into a linear and flat-looking Riemannian Manifold, and we must rely on the Euclidean Geometry (and gradient descent). We are descending towards the steepest descent direction from the euclidean distance point of view, which is not necessarily the real steepest descent direction for the Manifold. Going back to the paper example, we are not unbending the paper before tracing the straight line... The most common metric used for that goal is the FIM which is a statistical Matrix built on exploration of the neighborhood for a given configuration of a Neural Network: it describes the probability distribution generated by the model changes when the weights are slightly perturbed. The FIM will assign a high curvature/distance if a slight change in weights generate a wide change in the model's output. This FIM effectively becomes a sort of delta-discovery matrix where eigenvalues (vectors focussed on the single perpendicular directions) are minima suggest regions where the surface is flatter (eigenvalue small means each direction has small variations).

In [3] we can see that they use the formula:

$$f(x, w_t + du) \approx f(x, w_t) + J_{wt} \cdot du \quad (1)$$

to represent the output of a neural network configured with weights w_t when it receives a fixed input x . Here the Jacobian matrix tells us how much the output varied with respect to each and every weight of the network for the infinitesimal movement du in the weight space.

They then compute the performance changes in the model as

$$d(w_t, w_p) = |f(x, w_t) - f(x, w_p)|^2 = du^T (J_{wt}^T \cdot J_{wt}) du = du^T g_{wt} du \quad (2)$$

notice that the Jacobian multiplied itself becomes a symmetric matrix hence we get that the final component of the equation is a norm around the matrix g_{wt} and in particular given that m is the

number of outputs:

$$g_{ij}(x) = \sum_{k=1}^m \frac{\partial f_k(x, w)}{\partial w^i} \frac{\partial f_k(x, w)}{\partial w^j} \quad (3)$$

So now we can use the g matrix as the Riemannian Manifold metric for distancing and we can apply the properties of a Riemannian Manifold to the weight space through this metric.

1.4 Riemannian Manifold Traversal

Now that we have an adaptive metric for the Manifold, we can traverse the space and measure reliably the distance metric independently from the local curvature as we designed $g(x)$ to be a varying ruler over that curvatures. The path can be then computed using Riemann Integrals that is by summing each local contribution to the total path:

$$S(\gamma) = \int_0^1 \left\langle \frac{d\gamma(t)}{dt}, \frac{d\gamma(t)}{dt} \right\rangle_{\gamma(t)} dt \quad (4)$$

where the $\left\langle \frac{d\gamma(t)}{dt}, \frac{d\gamma(t)}{dt} \right\rangle_{\gamma(t)}$ is equivalent to $\frac{d\gamma(t)}{dt}^T g_{\gamma(t)} \frac{d\gamma(t)}{dt}$. The minimum of that function is the solution to the Geodesic Equation:

$$\frac{d^2 w^\eta}{dt^2} + \Gamma_{\mu\nu}^\eta \frac{dw^\mu}{dt} \frac{dw^\nu}{dt} = 0 \quad (5)$$

where $\Gamma_{\mu\nu}^\eta$ specifies the Christoffel symbols (intuitively those indicates how the g function variates along a selected set of directions) in [3] it is used to develop a gradient-descent-like algorithm that starts from w_t and traverses the weight space by moving towards the least steepest descending curve relatively to the objective encoded by w_t while also traversing the steepest descending curve accordingly to a secondary objective w_a by exploiting local corrections as follows:

$$\operatorname{argmin}_{\theta(w)} \langle \theta(w), \theta(w) \rangle_w - \beta \theta(w)^T (w_a - w) \quad \text{subject to } \theta(w)^T \theta(w) \leq 0.01 \quad (6)$$

This equation finds the argument that minimizes the difference hence that has minimal change in $\theta(w)$ that is the original problem that the network solves, while also maximizing the movement towards the $w_a - w$ vector that is the vector pointings towards w_a in the weight space, moving along the "straight" - tangent - directions according to the $\theta(w)$ variations built on g Riemannian Metric. After computing 6, they apply the correction:

$$\Delta w = \eta \frac{\theta(w)}{\|\theta(w)\|} \quad (7)$$

2 Learning The Weight Space

One of the core issues with trying to explore the weight space is that it explodes as the network becomes even slightly more complex. Dimensionality of this space is huge and highly suffers the curse of dimensionality problem. For this reason in [4] the authors try to solve this problem by hitting an interesting intuition:

a neural network is a sequential chain of computations in which deeper layers solely rely on previous ones. It is hence possible to tokenize the layers and work on fixed dimensional tokens

Using SANE they learn a weight space task-agnostic tokenization of the network. Assume, for instance, to have a DNN able to solve a task and that this neural network is made of a lot of layers: each layer applies a computation based on previous layers' outputs: first output is the input itself that comes into the first layer of the network, so we will have that the first layer will model a task-agnostic function that modifies the input using the given weights:

$$input \rightarrow blackbox_0(x) \rightarrow output_0 \Rightarrow f_0(w_0, x) \quad (8)$$

$$output_0 \rightarrow blackbox_1(x) \rightarrow output_1 \Rightarrow f_1(w_1, f_0(w_0, x)) \quad (9)$$

$$\dots \quad (10)$$

inductively: if $f_0(w_0, x)$ is a blackbox modelling a function that is task-agnostic also $f_1(w_1, x')$ is where $x' = f_0(w_0, x)$ so we can collect those two layers and consider them a single super-neuron that applies a blackbox function to the input $f_b(x)$. This grouping can be applied to any group and, for instance, can be applied to all layers in-between subsequent activation functions, so we can study the activation map of an input x as the result of $activation(f_b(x))$. **Notice that any type of grouping strategy can be used.**

2.1 The dimensionality problem

SANE is a Sequential Autoencoder for Neural Embeddings, but as all autoencoders there is a huge problem when dealing with dimensionality reduction: applying too much reduction often results in loss of information causing the inability of the autoencoder to reconstruct the original input correctly, plus we need to create $n \times m$ trainable parameters where n is the number of trainable parameters of the network, which is highly infeasible.

Now if we consider the grouping strategy stated before we can think of the DNN as a sequence of Super-Neurons that apply very complex computation and only the most relevant neurons will activate forwarding their outputs, creating multiple maps between input and output, each neuron in an hidden layer is completely independent from the other neurons in the same layer as it solely depends on the previous computations hence the network is endowed with the property of sequentiality where each layer is a block of the sequence. We can hence think of the computation as a sequence of $t_0 \rightarrow t_1 \rightarrow \dots \rightarrow t_n$, for a fixed t_i we can further separate weights as their paths are completely independent from the others in the same t_i as the neuron doesn't depend on the future steps (time-series-like computation) let alone on the future steps of other neurons in the same t_i . This is also shown experimentally by [1, 2]

2.2 Neurons Properties

Try thinking about the path a fixed input x takes into the DNN: when it enters the first layer of the network will apply some operations to the input firing a very specific set of outputs (one for each neuron in the successive layer) that only depends on the neuron's weights. All neurons in the same layer will apply the same transformation f_l with different weights, so each neuron in the same layer will create differences in how to treat the input.

Definition 1. *Moving a Neuron means changing it's position in the layer without affecting the final output of the network*

In order to not affect the output, given that the input x is fixed, we need to swap both ingoing and outgoing weights that are the input weights to the current neuron, which will change the output ordering, hence also the input weights to each neuron of the next layer must be swapped accordingly so that we just change the sums ordering without changing the overall output to the sequent layer. We could, thus, say that:

Property 1. *A Neuron can be identified in the weight space as a specific combination of ordered ingoing and outgoing weights, the position of the neuron - ingoing weights + applied function + outgoing weights - does not affect the network output*

Property 2. *A Layer is a set of specific neurons - identified as in 2.2 - and encodes a specific set of functions where each function is the input $\rightarrow$ output map happening in the specific neuron. The ordering of those functions do not affect the final output of the network*

So if we now apply those concepts to a Super-Neuron we can view it as a sequence of layers that doesn't change as long as we do not change the sequence ordering as:

$$SN_i = w_{i-n} \circ w_{i-n+1} \circ \dots \circ w_i \circ a_i \quad (11)$$

where w_i are the ordered weights in the i th layer and a_i is the activation function applied at the end of the super-neuron. Equivalently to 11:

$$SN_i = f_{i-n} \circ f_{i-n+1} \circ \dots \circ f_i \circ a_i \quad (12)$$

Being that swapping ingoing weights ordering to the overall function on each layer doesn't affect the function at all as long as we change the outgoing weights accordingly, each function in the concatenation will stay the same independently from neurons position. We can thus

- *Identify a neuron in a layer as the ordered combination of ingoing and outgoing weights*
- *Identify a layer by it's position in the sequence and the set - sets are not order-relevant - of neurons that belongs to it*

Finally, we can state that:

Definition 2. *Moving a super-neuron means to move only the relative neurons in the final $f_i \circ a_i$ layer*

Given a change in any of the initial layers infact wouldn't impact the final input to the subsequent layer in the chain, any change made to the intial layers are, thus, completely irrelevant with moving the Super-Neuron, which moval reduces to the moval of the relative neuron in the final layer to which the activation is applied.

As the only difference between DNNs and CNNs are the way "neurons" behave, therefore the function they apply, the same concepts can be also translated to them, where each neuron of a layer is the kernel and each layer is a convolution of those kernels against the input, convolution is just a different behavior of the neurons that doesn't alter the properties of the network, hence, also CNNs inherit the properties discussed.

3 Generating Hyper-representations of NN

Making the assumptions of the previous section, it is possible to simplify greatly the weight space exploration as now we know that we can group weights into neurons, neurons into layers and layers into super-neurons. We can establish a top-down approach to explore each individually, greatly reducing weights redundancy and search space.

Imagine, for instance, to have a very complex CNN, with multiple convolutions + batch normalization + residual blocks before each activation layer: we could use the in-between-activations grouping strategy to get Super-Neurons and encode those Super-Neurons by exploiting the input $\rightarrow$ output mapping, so we can now use AutoEncoders to generate an embedding of those and reconstruct/generate new task-agnostic Super-Neurons exploiting the latent space language of the AutoEncoders or even of Neural Transformers. Infact, we can do much more: we can even "open" a Super-Neuron and explore it individually to make aimed changes to its behavior by exploring the layer sequence that generates them, moreover we could even explore single layers using the very same approach used for Super-Neurons and applying them to simple neurons.

The implications of such a framework could bring enormous benefits to the Machine Learning research:

- **Automated Architecture Design:** Neural Architecture Search (NAS) often involves discrete search spaces and computationally heavy evaluations, here we can basically learn a language able to be translated into entire sequences of layers (the Super Neurons).
- **Interpretability and Understanding:** XAI is also a fundamental aspect of Machine Learning, exploring the entirety of the weights space often leads to miss-interpretations and computational difficulties, symmetries in the weights space though, due to the existence of neurons and their sequentiality, can lead to much more effective analysis as well as a top-down debugging of the networks
- **Network Generation and Evolution:** Given a neuron is just a tensor, and a super-neuron a sequence of organized neurons, we can explore and generate new model's for specific task with approaches that were unthinkable before: Genetic exploration of Super-Neurons space as well as neurons space, language translation exploiting autoencoders and transformers to relate multimedial-based input with NN models and more.
- **Transferability:** Once a Neuron gets used and solves a specific task, it could also be reused in its same configuration within other neural networks, which is basically what Genetic Exploration of Super-Neurons would do, allowing flexibility and transferability of neurons.

References

- [1] C. H. Martin and M. W. Mahoney. Rethinking generalization requires revisiting old ideas: statistical mechanics approaches and complex learning behavior, 2019.
- [2] C. H. Martin, T. Peng, and M. W. Mahoney. Predicting trends in the quality of state-of-the-art neural networks without access to training or testing data. *Nature Communications*, 12(1), 2021.
- [3] G. Raghavan and M. Thomson. Solving hybrid machine learning tasks by traversing weight space geodesics, 2021.

- [4] K. Schürholt, M. W. Mahoney, and D. Borth. Towards scalable and versatile weight space learning, 2024.